EDA using Bivariate and Multivariate Analysis

100 Days of Machine Learning | Day 21 | CampusX

1. Introduction

After performing Univariate Analysis (analyzing one column at a time), the next logical step in Exploratory Data Analysis (EDA) is to analyze the relationships between multiple columns.

- **Bivariate Analysis:** Analyzing exactly two columns together.
- **Multivariate Analysis:** Analyzing more than two columns simultaneously.

The type of plot you choose depends entirely on the data types of the columns you are comparing: **Numerical vs Numerical**, **Numerical vs Categorical**, or **Categorical vs Categorical**.

2. Numerical vs Numerical Analysis

Scatter Plot (`sns.scatterplot`)

The best tool to find the relationship (linear, non-linear, or no relation) between two continuous numerical values.

```
import seaborn as sns
import matplotlib.pyplot as plt

# Bivariate: Total Bill vs Tip
sns.scatterplot(x=tips['total_bill'], y=tips['tip'])

# Multivariate: Adding a Categorical Column (hue)
# Colors the dots based on gender
sns.scatterplot(x=tips['total_bill'], y=tips['tip'], hue=tips['sex'])

# Multivariate: Adding Style and Size
# Different shapes for smokers, different sizes based on group size
sns.scatterplot(x=tips['total_bill'], y=tips['tip'], hue=tips['sex'],
style=tips['smoker'], size=tips['size'])
plt.show()
```

Line Plot (`sns.lineplot`)

A special case of a scatterplot where the points are connected by a line. It is strictly used when the X-axis represents a **Time-based quantity** (Years, Months, Dates, etc.).

```
# Group flights by year and sum passengers
new = flights.groupby('year').sum().reset_index()

# Plot the trend over time
sns.lineplot(x=new['year'], y=new['passengers'])
plt.show()
```

3. Numerical vs Categorical Analysis

Bar Plot (`sns.barplot`)

Used to find aggregate values (like average/mean, sum) of a numerical column grouped by a categorical column. By default, seaborn's barplot calculates the **Mean**.

```
# X-axis: Categorical (Passenger Class), Y-axis: Numerical (Age)
sns.barplot(x=titanic['Pclass'], y=titanic['Age'])

# Multivariate: Adding hue
sns.barplot(x=titanic['Pclass'], y=titanic['Age'], hue=titanic['Sex'])
plt.show()
```

Box Plot (`sns.boxplot`)

Used to compare the 5-number summary (min, Q1, median, Q3, max) and outliers of a numerical column across different categories.

```
# Compare the age distribution between Males and Females
sns.boxplot(x=titanic['Sex'], y=titanic['Age'])

# Multivariate: Adding survival status
sns.boxplot(x=titanic['Sex'], y=titanic['Age'], hue=titanic['Survived'])
plt.show()
```

Distplot (KDE Plot Overlay)

You can plot multiple Probability Density Functions (PDFs) on the same graph to compare distributions across categories.

```
# Plot age distribution of people who died (Survived == 0)
sns.distplot(titanic[titanic['Survived'] == 0]['Age'], hist=False, label='Died')

# Plot age distribution of people who survived (Survived == 1)
sns.distplot(titanic[titanic['Survived'] == 1]['Age'], hist=False,
label='Survived')
plt.show()
```

Example Insight: In the Titanic dataset, overlaying these plots reveals that young children had a higher probability of surviving than dying, while older age groups had a higher probability of dying.

4. Categorical vs Categorical Analysis

Heatmap (`sns.heatmap`)

Used to visualize a matrix/grid where color intensity represents numerical values. Often used in combination with pandas `crosstab` or `pivot_table`.

```
import pandas as pd

# 1. Create a crosstab between two categorical columns
cross_tab = pd.crosstab(titanic['Pclass'], titanic['Survived'])

# 2. Visualize using Heatmap
sns.heatmap(cross_tab)
plt.show()
```

Cluster Map (`sns.clustermap`)

Similar to a heatmap, but it automatically uses Hierarchical Clustering to group rows and columns that show similar behavior together.

```
# Groups similar passenger classes and survival outcomes together
sns.clustermap(pd.crosstab(titanic['Parch'], titanic['Survived']))
plt.show()
```

5. Automated Multivariate Analysis

Pair Plot (`sns.pairplot`)

If a dataset has many numerical columns, manually plotting scatterplots for every possible pair is tedious. `pairplot()` automatically detects all numerical columns and creates a massive grid of scatterplots for every possible combination.

```
# Automatically plots all numerical relationships in the iris dataset
sns.pairplot(iris, hue='species')
plt.show()
```